DATA 255 Deep Learning Technologies

Sept 11, 2025

Taehee Jeong, Ph.D.

SJSU SAN JOSÉ STATE UNIVERSITY

Today's Topics

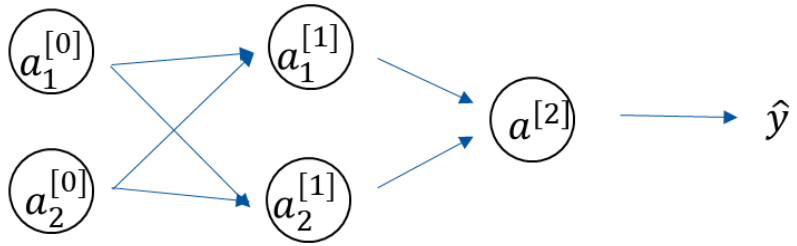
- Model Optimization Procedure
- Gradient Descent
- Backward Pass for binary classification

Model optimizing procedure

1. Initialize parameters
2. Run the optimization loop
    1. Forward propagation to compute the loss function
    2. Backward propagation to compute the gradients with respect to the loss function
    3. Using the gradients, update your parameter with the gradient descent update rule
3. Return the learned parameters

Source: deep learning, Andrew Ng

Forward propagation



1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

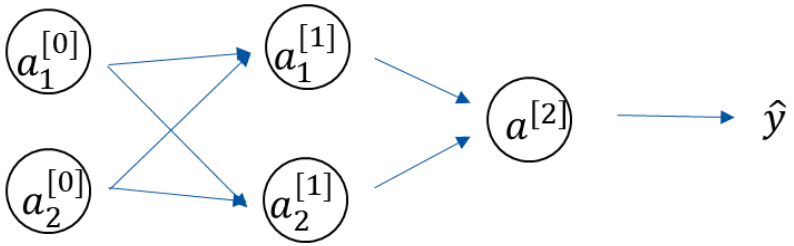
2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

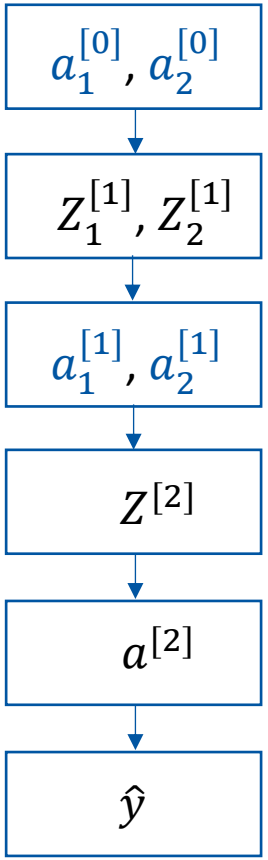
$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = \begin{cases} 1 \; if \; a^{[2]} \geq 0.5 \\ 0 \; if \; a^{[2]} < 0.5 \end{cases}$$

Forward propagation



Input layer Hidden layer Output layer



Gradient Descent

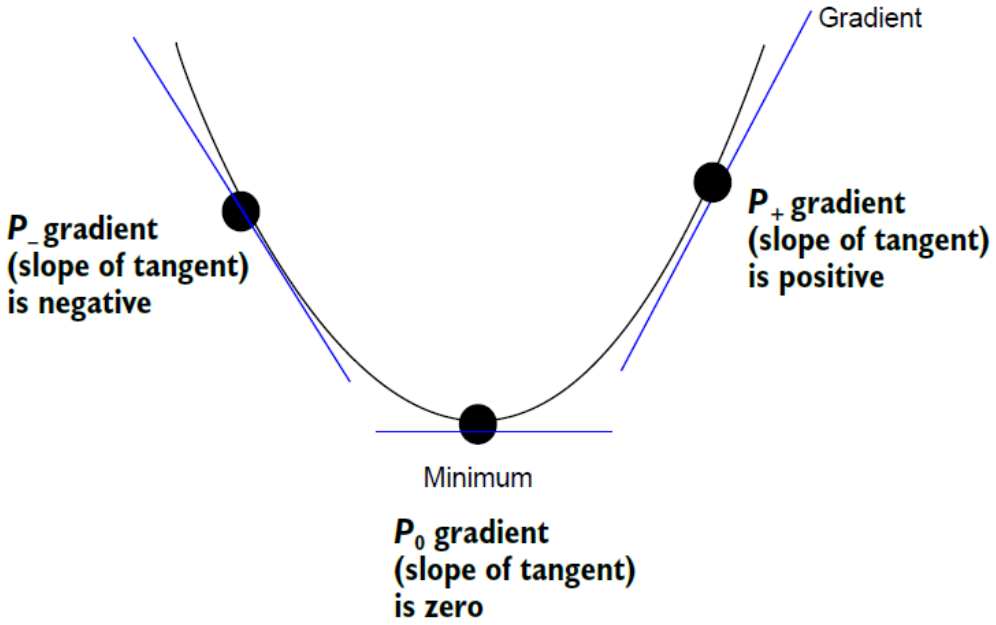
Find optimum W to minimize J

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

$$J = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}, y)$$

- *W : weight or Model parameters*
- *J : cost function*
- *L: loss function(cross entropy)*
- α: *learning rate or step size*
- $\frac{\partial J}{\partial w}$: gradient

Gradient is zero at the minimum

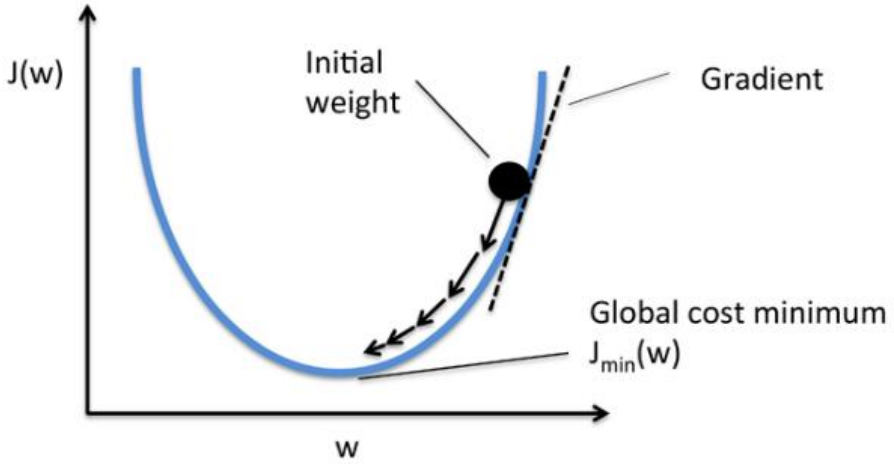


Any optimum (that is, maximum or minimum) of a function is a point of inflection.
This means the function turns around at the optimum point.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Gradient descent method

- To find the weights that minimize a cost function, gradient descent is commonly used as an optimization algorithm.
- Gradient(vector of partial derivatives) gives the input direction in which the function **decreases**
  - Step0: pick a random starting point
  - Step1: compute the gradient
  - Step2: take a small step in the direction of the gradient
  - Step3: repeat with the new starting point



Source: https://www.bogotobogo.com/python/scikit-learn/scikit-learn_batch-gradient-descent-versus-stochastic-gradient-descent

Backward propagation

- The derivative of the loss in forward pass needs to be computed with respect to the weights in all layers in the backwards pass.
- The main goal of the backward propagation is to learn the gradient of the loss function with respect to the different weights by using the chain rule.
- These gradients are used to update the weights.
- Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward pass.

Source: Neural Networks and Deep Learning, Charu C. Aggarwal

Backpropagation

Backpropagation is a way of computing gradients of expressions through recursive application of chain rule.

Derivative formula

Product rule

$h(x) = f(x) \cdot g(x)$

$$\frac{d}{dx}\, h(x) = \left[\frac{d}{dx} f(x)\right] \cdot g(x) + f(x) \cdot \left[\frac{d}{dx} g(x)\right]$$

$h(x) = f(g(x))$

$$\frac{d}{dx}[f(g(x))] = f'\big(g(x)\big) \cdot g'\,(x)$$

Chain rule

If $y = f(u)$, where $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Derivative formula

Product rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{\partial h}{\partial x} = f' \cdot g + f \cdot g'$$

$$h(x) = \frac{f(x)}{g(x)}$$

$$h = f \cdot g^{-1}$$

$$\frac{\partial h}{\partial x} = f' \cdot g^{-1} + f \cdot \{-g^{-2}\} \cdot g'$$

$$= \frac{f'}{g} - \frac{f \cdot g'}{g^2}$$

$$= \frac{f' \cdot g - f \cdot g'}{g^2}$$

Derivatives of Sigmoid function

$$g(z) = \frac{1}{1 + \exp(-z)}$$

$$\frac{d}{dx} g(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$g'(z) = g(z)(1 - g(z))$$

Derivatives of Tanh function

$$g(z) = tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

$$g'(z) = 1 - g(z)^{2}$$

Derivatives of ReLU and Leaky ReLU

ReLU

$$a = g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 0 & if\ z < 0 \\ 1 & if\ z \geq 0 \end{cases}$$

Leaky ReLU

$$a = g(z) = \max(0.01z, z)$$

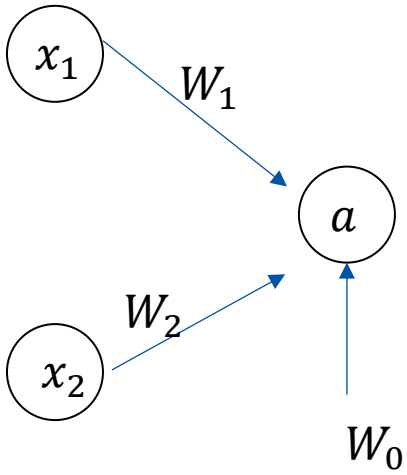
$$g'(z) = \begin{cases} 0.01 & if\ z < 0 \\ 1 & if\ z \geq 0 \end{cases}$$

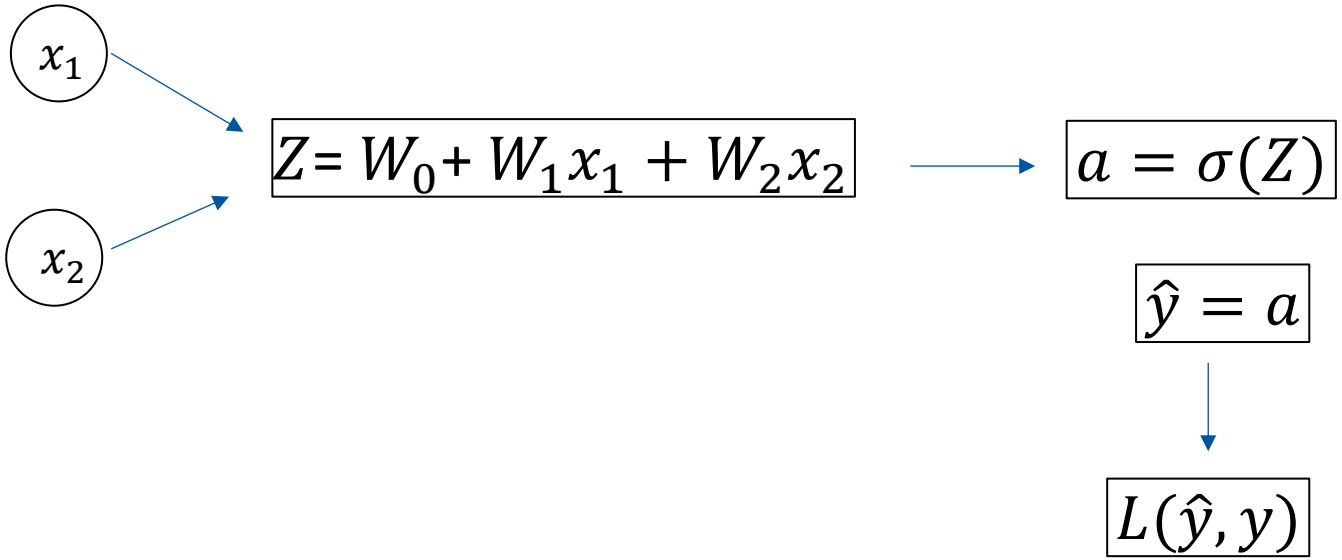
Derivatives of ELU

$$a = ELU(z) = \begin{cases} \alpha(\exp(z) - 1) & if\ z < 0 \\ z & if\ z \geq 0 \end{cases}$$

$$g'(z) = \begin{cases} \alpha \exp(z) & if\ z < 0 \\ 1 & if\ z \geq 0 \end{cases}$$

Logistic regression





$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

Derivative of binary cross entropy

$$L(\hat{y}, y) = -[y\log\hat{y} + (1-y)\log(1-\hat{y})]$$

$$\frac{\partial L}{\partial W_0} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial Z}\frac{\partial Z}{\partial W_0}$$

$$\frac{\partial L}{\partial W_0} = \left[-\frac{y}{\hat{y}} + \frac{1-y}{(1-\hat{y})}\right][\hat{y} * (1-\hat{y})] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = [-y(1-\hat{y}) + \hat{y}(1-y)] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = (\hat{y} - y) \cdot 1$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$\hat{y} = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(\hat{y}, y) = -[y\log\hat{y} + (1-y)\log(1-\hat{y})]$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial Z}\frac{\partial Z}{\partial W_1}$$

$$\frac{\partial L}{\partial W_1} = \left[-\frac{y}{\hat{y}} + \frac{1-y}{(1-\hat{y})}\right][\hat{y} * (1-\hat{y})] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = [-y(1-\hat{y}) + \hat{y}(1-y)] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = (\hat{y} - y) \cdot x_1$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$\hat{y} = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(\hat{y}, y) = -[y\log\hat{y} + (1-y)\log(1-\hat{y})]$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial Z}\frac{\partial Z}{\partial W_2}$$

$$\frac{\partial L}{\partial W_2} = \left[-\frac{y}{\hat{y}} + \frac{1-y}{(1-\hat{y})}\right][\hat{y} * (1-\hat{y})] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = [-y(1-\hat{y}) + \hat{y}(1-y)] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = (\hat{y} - y) \cdot x_2$$

$$\frac{d}{dx}\, log(x) = \frac{1}{\mathrm{x}}$$

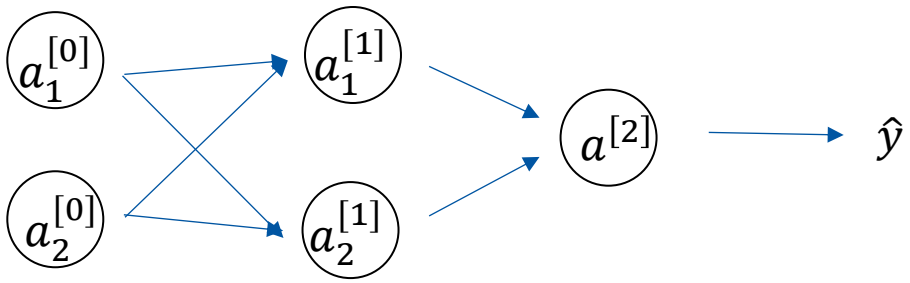
$$\frac{d}{dx}\, log(1-x) = \frac{-1}{1-\mathrm{x}}$$

$$\hat{y} = \sigma(z)$$

$$\frac{d}{dz}\sigma(z) = \sigma(z) * [1-\sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Neural Network: 1 hidden layer



Input layer Hidden layer Output layer

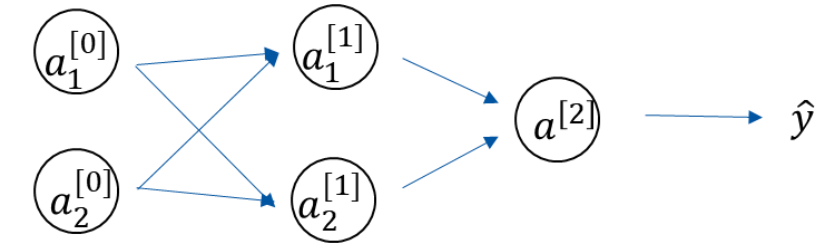
Parameters: $W^{[1]}$, $W^{[2]}$

Cost function: $L(W^{[1]}, W^{[2]})$

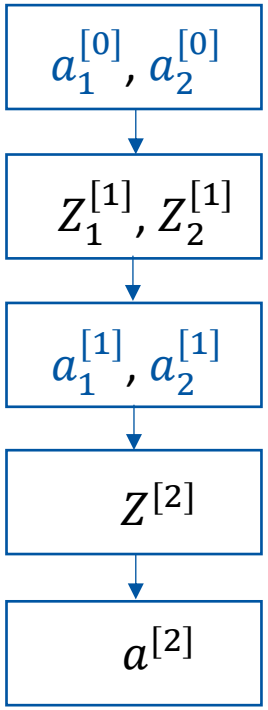
$$= \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}, y)$$

$$= \frac{1}{m}\sum_{i=1}^{m} L(a^{[2]}, y)$$

Forward propagation



Input layer Hidden layer Output layer



Derivative of $Z^{[2]}$

backpropagation for binary entropy loss

$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

$$L(\hat{y}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}}$$

$$dZ^{[2]} = (a^{[2]} - y)$$

**Element wise multiplication*

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$
	$\hat{y} = a^{[2]}$

Derivative of $W_0^{[2]}$

backpropagation for binary entropy loss

$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

$$L(\hat{y}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_0^{[2]}} = dW_0^{[2]}$$

$$dW_0^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}}$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

**Element wise multiplication*

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$
	$\hat{y} = a^{[2]}$

Derivative of $W_1^{[2]}$

backpropagation for binary classification

$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

$$L(\hat{y}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(\hat{y}, y)}{\partial W_1^{[2]}} = dW_1^{[2]}$$

$$dW_1^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}}$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

**Element wise multiplication*

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$
	$\hat{y} = a^{[2]}$

Derivative of $W_2^{[2]}$

backpropagation for binary classification

$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

$$L(\hat{y}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(\hat{y}, y)}{\partial W_2^{[2]}} = dW_2^{[2]}$$

$$dW_2^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

**Element wise multiplication*

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$
	$\hat{y} = a^{[2]}$

Derivative of Parameters in 2nd layer

backpropagation for binary entropy loss

$$L(\hat{y}, y) = -[y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})]$$

$$dZ^{[2]} = a^{[2]} - y$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

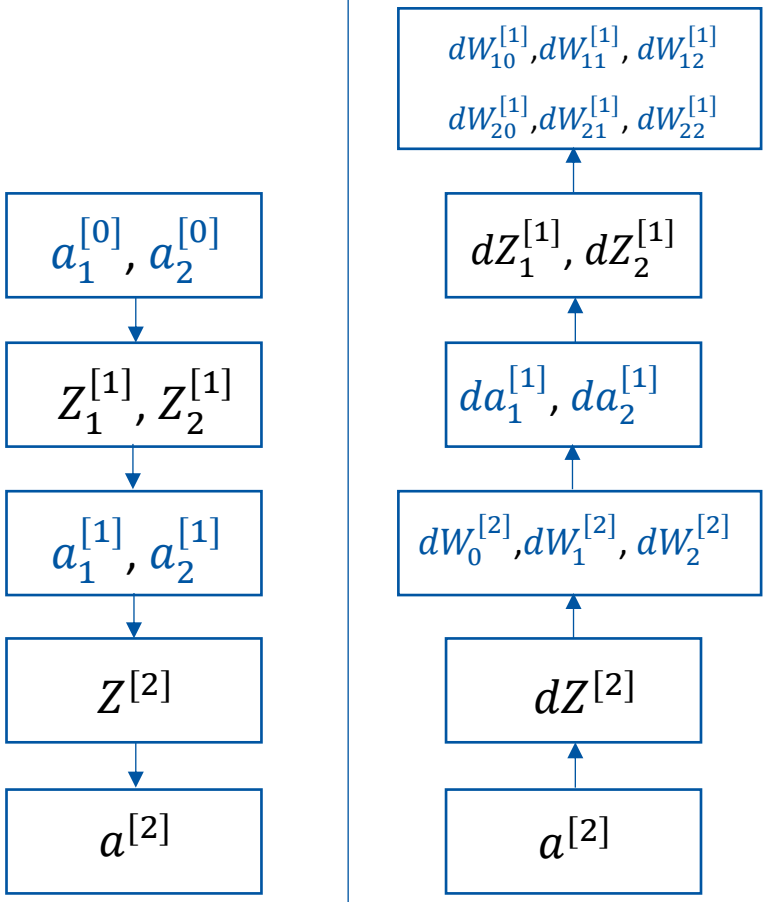
$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

1st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$
	$\hat{y} = a^{[2]}$

Forward propagation

Backward propagation



1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$\hat{y} = a^{[2]}$$

Cache forward pass variables

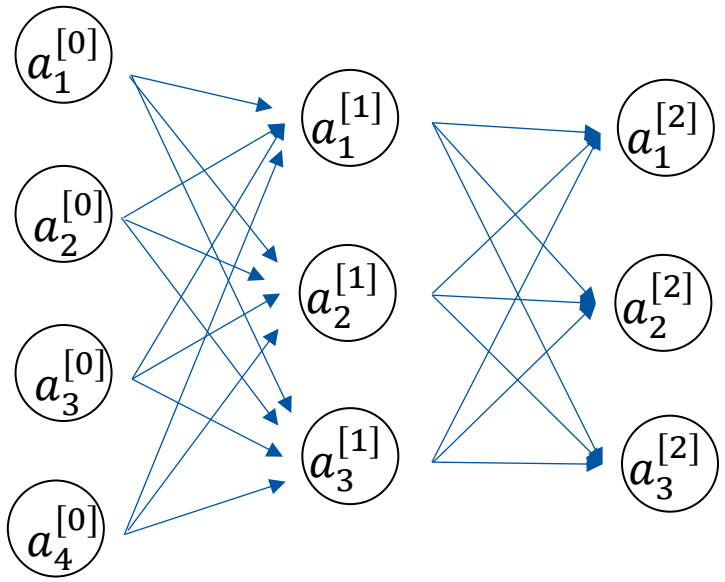
To compute the backward pass,

it is very helpful to have some of the variables that were used in the backward pass or forward pass.

In practice you want to structure your code so that you cache these variables,

and so that they are available during backpropagation.

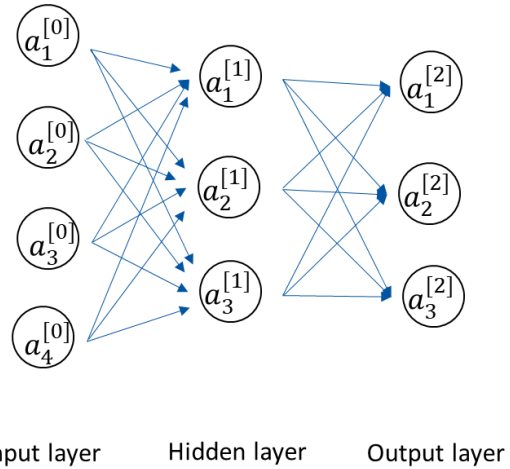
Neural Network: 3 output neurons



Input layer Hidden layer Output layer

Activation function of the last layer is *Softmax*.

Forward propagation



1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]} + W_{13}^{[1]} a_3^{[0]} + W_{14}^{[1]} a_4^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]} + W_{23}^{[1]} a_3^{[0]} + W_{24}^{[1]} a_4^{[0]}$$

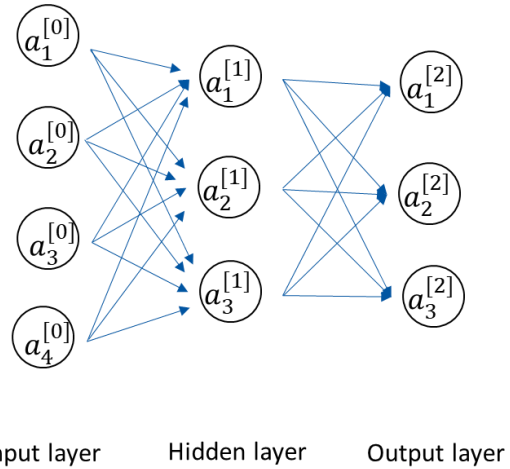
$$Z_3^{[1]} = W_{30}^{[1]} + W_{31}^{[1]} a_1^{[0]} + W_{32}^{[1]} a_2^{[0]} + W_{33}^{[1]} a_3^{[0]} + W_{34}^{[1]} a_4^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$a_3^{[1]} = \sigma(Z_3^{[1]})$$

Forward propagation



2nd layer

$$Z_1^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} a_1^{[1]} + W_{12}^{[2]} a_2^{[1]} + W_{13}^{[2]} a_3^{[1]}$$

$$Z_2^{[2]} = W_{20}^{[2]} + W_{21}^{[2]} a_1^{[1]} + W_{22}^{[2]} a_2^{[1]} + W_{23}^{[2]} a_3^{[1]}$$

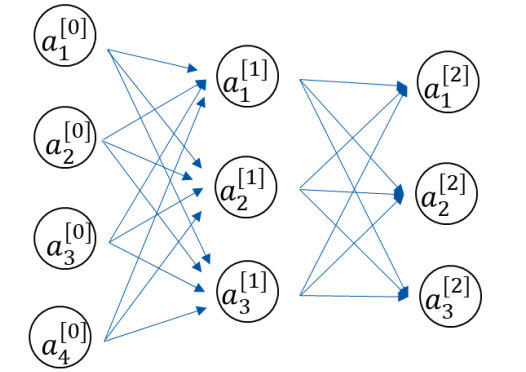
$$Z_3^{[2]} = W_{30}^{[2]} + W_{31}^{[2]} a_1^{[1]} + W_{32}^{[2]} a_2^{[1]} + W_{33}^{[2]} a_3^{[1]}$$

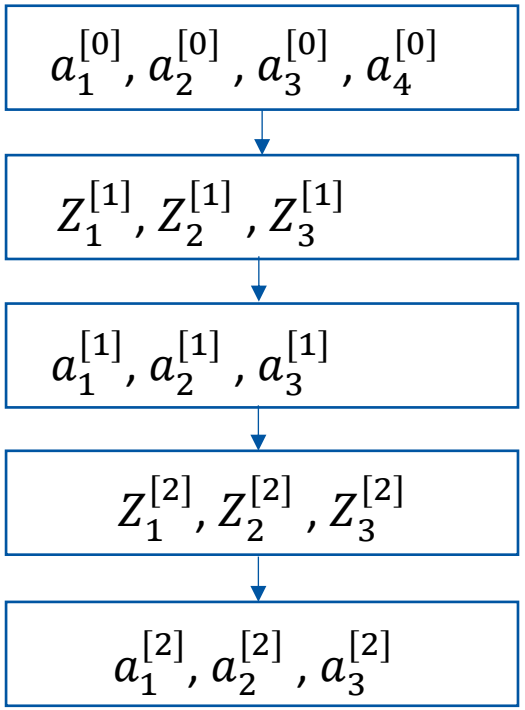
$$a_1^{[2]} = S(Z_1^{[2]})$$

$$a_2^{[2]} = S(Z_2^{[2]})$$

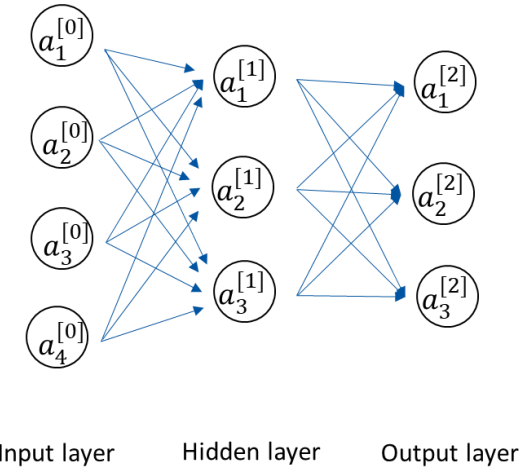
$$a_3^{[2]} = S(Z_3^{[2]})$$

Forward propagation





Derivative of $a^{[2]}$



backpropagation for cross entropy loss

$$L(\hat{y}, y) = -[y_1 * \log \widehat{y_1} + y_2 * \log \widehat{y_2} + y_3 * \log \widehat{y_3}]$$

$$L(\hat{y}, y) = -[y_1 * \log\left(a_1^{[2]}\right) + y_2 * \log\left(a_2^{[2]}\right) + y_2 * \log(a_3^{[2]})]$$

Activation function of output layer is softmax

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\sum_{j=1}^{K} \exp(Z_j^{[2]})}$$

$$a_1^{[2]} = \frac{\exp(Z_1^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

$$a_2^{[2]} = \frac{\exp(Z_2^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

$$a_3^{[2]} = \frac{\exp(Z_3^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

Derivative of $softmax$

$$S(Z_1) = \frac{\exp(Z_1)}{\exp(Z_1) + \exp(Z_2) + \exp(Z_3)}$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i)) \qquad if\ i = j$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j) \qquad if\ i \neq j$$

Chain rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{d}{dx} h(x) = \left[\frac{d}{dx} f(x)\right] \cdot g(x) + f(x) \cdot \left[\frac{d}{dx} g(x)\right]$$

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} e^{-x} = -e^{-x}$$

$$a_1^{[2]} = S(Z_1^{[2]}) = \frac{\exp(Z_1^{[2]})}{\exp\left(Z_1^{[2]}\right) + \exp\left(Z_2^{[2]}\right) + \exp\left(Z_3^{[2]}\right)}$$

Derivative of $Z^{[2]}$

$$\frac{\partial S(Z_j)}{\partial Z_i} = S(Z_i) \times (1 - S(Z_i))$$

$$\frac{\partial S(Z_j)}{\partial Z_i} = -S(Z_i) \times S(Z_j)$$

$$L(\hat{y}, y) = -[y_1 * \log\left(a_1^{[2]}\right) + y_2 * \log\left(a_2^{[2]}\right) + y_3 * \log(a_3^{[2]})]$$

$$L(\hat{y}, y) = -[y_1 * \log\left(S(Z_1^{[2]})\right) + y_2 * \log\left(S(Z_2^{[2]})\right) + y_3 * \log(S(Z_3^{[2]})]$$

$$dZ_1^{[2]} = -\frac{y_1}{a_1^{[2]}} \cdot \frac{\partial S(Z_1^{[2]})}{\partial Z_1^{[2]}} - \frac{y_2}{a_2^{[2]}} \cdot \frac{\partial S(Z_2^{[2]})}{\partial Z_1^{[2]}} - \frac{y_3}{a_3^{[2]}} \cdot \frac{\partial S(Z_3^{[2]})}{\partial Z_1^{[2]}}$$

$$dZ_1^{[2]} = -y_1 \cdot (1 - a_1^{[2]}) + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + y_1 \cdot a_1^{[2]} + y_2 \cdot a_1^{[2]} + y_3 \cdot a_1^{[2]}$$

$$dZ_1^{[2]} = -y_1 + a_1^{[2]}$$

$$dZ_1^{[2]} = a_1^{[2]} - y_1$$

Forward propagation

$a_1^{[0]}, a_2^{[0]}, a_3^{[0]}, a_4^{[0]}$

$Z_1^{[1]}, Z_2^{[1]}, Z_3^{[1]}$

$a_1^{[1]}, a_2^{[1]}, a_3^{[1]}$

$Z_1^{[2]}, Z_2^{[2]}, Z_3^{[2]}$

$a_1^{[2]}, a_2^{[2]}, a_3^{[2]}$

Backward propagation

$dW_{10}^{[2]}, dW_{11}^{[2]}, dW_{12}^{[2]}, dW_{13}^{[2]}$

$dW_{20}^{[2]}, dW_{21}^{[2]}, dW_{22}^{[2]}, dW_{23}^{[2]}$

$dW_{30}^{[2]}, dW_{31}^{[2]}, dW_{32}^{[2]}, dW_{33}^{[2]}$

$dZ_1^{[2]}, dZ_2^{[2]}, dZ_3^{[2]}$

$a_1^{[2]}, a_2^{[2]}, a_3^{[2]}$

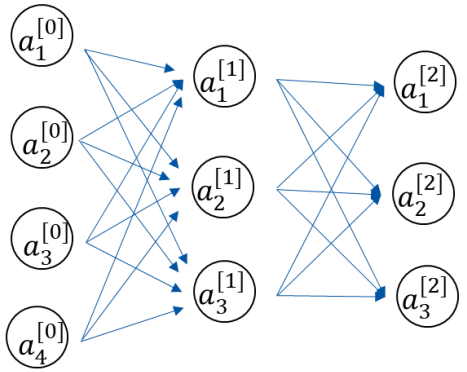
$dW_{10}^{[1]}, dW_{11}^{[1]}, dW_{12}^{[1]}, dW_{13}^{[1]}, dW_{14}^{[1]}$

$dW_{20}^{[1]}, dW_{21}^{[1]}, dW_{22}^{[1]}, dW_{23}^{[1]}, dW_{24}^{[1]}$

$dW_{30}^{[1]}, dW_{31}^{[1]}, dW_{32}^{[1]}, dW_{33}^{[1]}, dW_{34}^{[1]}$

$dZ_1^{[1]}, dZ_2^{[1]}, dZ_3^{[1]}$

$da_1^{[1]}, da_2^{[1]}, da_3^{[1]}$



Autograd: PyTorch automatic gradient computation

- For a specific model architecture, we computed the gradient using calculus.
- This approach does not scale to more complex models with millions of weights and perhaps nonlinear complex functions.
- For scalability, we use an automatic differentiation software library like PyTorch Autograd.
- Users of the library need not worry about how to compute the gradients—they just construct the model function.
- Once the function is specified, PyTorch figures out how to compute its gradient through the Autograd technology.

Source: Math and Architectures of Deep Learning, Krishenedu Chauhury